

MR

Multi-Edit Revisited

Userhandbuch v1.0

Projekt: <https://github.com/ebeneezer/MR>

22. Juni 2026

*„Software and cathedrals are much the same –
first we build them, then we pray.“*
— Sam Redwine

Inhaltsverzeichnis

1	Einführung	1
1.1	Was ist MR?	1
1.2	Kernphilosophie	1
1.3	Hauptfeatures auf einen Blick	1
2	Installation	3
2.1	Systemvoraussetzungen	3
2.2	Installation aus dem Quellcode	3
2.3	Erster Start	3
3	Die MRMAC-Makrosprache	5
3.1	Einführung in MRMAC	5
3.1.1	Der Makro-Compiler	5
3.1.2	Der Makro-Coprozessor	5
3.1.3	Beispiel: Einfaches MRMAC-Makro	5
3.2	Makro-Manager	6
4	Editieren wie ein Profi	7
4.1	Umgang mit großen Dateien	7
4.2	Blocktypen	7
4.3	Code-Faltung und Smart-Indenting	7
4.4	Die Minimap	8
5	Fenster-Management	9
5.1	Der Fenster-Manager	9
5.2	Virtuelle Desktops	9
5.3	Inter-Window Copy & Paste	9
6	Workspaces und Projekte	10
6.1	Workspace-Management	10
6.2	Profile	10
6.3	Farbthemen	10
7	Suche und Ersetzung	11
7.1	Reguläre Ausdrücke	11
7.2	Rekursive Multifile-Suche	11

8	Compiler-Integration	12
8.1	Compiler-Profile	12
8.2	Fehler-Tracking	12
9	LSP-Integration	13
9.1	Language Server Protocol	13
9.2	LSP-Konfiguration	13
10	Tastatur und Keymapping	14
10.1	Key-Mapping-Manager	14
10.2	Emulationsmodi	14
11	Erweiterte Funktionen	15
11.1	Drucken und Export	15
11.2	Pipes und Shell-Kommandos	15
11.3	Automatische Sprachdetektion	15
12	Tastaturreferenz (Auszug)	16
13	Fehlerbehebung	17
13.1	Häufige Probleme	17
14	Technische Architektur	18
14.1	Verwendete Technologien	18
14.2	Der Coprozessor im Detail	18
A	MRMAC-Sprachreferenz (Kurzfassung)	19
A.1	Grundlegende Syntax	19
A.2	Editor-Kommandos (Auswahl)	19
B	Lizenz und Danksagungen	21
B.1	Zitate, die uns inspirierten	21
B.2	Projekt	21

Abbildungsverzeichnis

Kapitel 1

Einführung

1.1 Was ist MR?

MR (Multi-Edit Revisited) ist eine Neuimplementierung des klassischen Programmier-Editors **Multi-Edit** von American Cybernetics für moderne Linux-Terminals. Nachdem American Cybernetics 2020 den Betrieb einstellte und die Entwicklung der TUI-Version bereits Jahre zuvor stoppte, wurde MR als Open-Source-Projekt ins Leben gerufen, um dieses legendäre Tool für die heutige Zeit wiederzubeleben.

Hinweis: MR ist ein *terminalbasierter* Editor (TUI), der speziell für Linux entwickelt wurde. Er nutzt `ncursesw` und ist vollständig UTF-8-fähig.

1.2 Kernphilosophie

MR verkörpert die Tugenden eines wahren Programmierers nach Larry Wall:

„The three chief virtues of a programmer are: Laziness, Impatience and Hubris.“

Das bedeutet konkret:

- **Faule** Programmierer schreiben Code, der Arbeit spart – MR automatisiert durch Makros
- **Ungeduldige** Programmierer erwarten sofortige Ergebnisse – MR lädt 1 GB Text in unter 1 Sekunde
- **Stolze** Programmierer wollen perfekten Code – MR bietet professionelle Werkzeuge

1.3 Hauptfeatures auf einen Blick

- ✓ Eigene Makrosprache **MRMAC** (kompatibel zu MEMAC)
- ✓ Paralleler Makro-Coprozessor mit mehreren Compute-Lanes
- ✓ Bearbeitung von Dateien **größer als der Systemspeicher**

- ✓ Lädt 1 GB Text in unter 1 Sekunde
- ✓ Volltext-Indexierung in unter 800 ms
- ✓ Automatisches Syntax-Highlighting für alle gängigen Sprachen
- ✓ Code-Faltung, Smart-Indenting, Minimap
- ✓ Virtuelle Desktops und Workspaces
- ✓ PCRE2 Regex-Unterstützung
- ✓ LSP-Integration
- ✓ Compiler-Profile mit automatischem Fehler-Tracking

Kapitel 2

Installation

2.1 Systemvoraussetzungen

Komponente	Anforderung
Betriebssystem	Linux (64-Bit empfohlen)
Terminal	UTF-8-fähig mit ncursesw-Unterstützung
Speicher	Abhängig von Dateigröße (Piecetable-Technologie)
Abhängigkeiten	ncursesw, PCRE2, TVISION (mitgeliefert)

2.2 Installation aus dem Quellcode

```
1 # Repository klonen
2 git clone https://github.com/ebeneezer/MR
3 cd MR
4
5 # Abhängigkeiten installieren (Debian/Ubuntu)
6 sudo apt install build-essential cmake libncursesw5-dev libpcre2-
   dev
7
8 # Bauen
9 mkdir build && cd build
10 cmake ..
11 make -j$(nproc)
12
13 # Optional: Systemweit installieren
14 sudo make install
```

2.3 Erster Start

Starten Sie MR einfach durch Eingabe von:

```
1 ./mr
```

Oder mit einer zu öffnenden Datei:


```
1 ./mr meine_datei.c
```

Kapitel 3

Die MRMAC-Makrosprache

3.1 Einführung in MRMAC

MRMAC ist das Herzstück von MR – eine mächtige Skriptsprache zur Editor-Automatisierung. Sie ist rückwärtskompatibel zum originalen MEMAC-Dialekt von Multi-Edit, wurde aber für moderne Systeme erweitert.

3.1.1 Der Makro-Compiler

MRMAC-Makros werden **in-RAM** kompiliert:

- **Vorkompiliert:** Für maximale Geschwindigkeit beim Laden
- **On-Demand:** Bei Bedarf zur Laufzeit kompiliert

3.1.2 Der Makro-Coprozessor

MR verfügt über einen eingebauten Coprozessor für MRMAC-Bytecode-Makros mit folgenden Eigenschaften:

- Manipulation von Text **parallel** zum Benutzer im selben Fenster
- Unterstützung mehrerer paralleler Makro-Jobs
- **Vier Compute-Lanes:**
 1. I/O – Ein-/Ausgabeoperationen
 2. COMPUTE – Berechnungen
 3. MACRO – Makro-Ausführung
 4. MINIMAP – Minimap-Aktualisierung

3.1.3 Beispiel: Einfaches MRMAC-Makro

```
1 // Ein einfaches MRMAC-Makro
2 macro HelloWorld()
3     InsertText("Hello from MRMAC!")
4     NewLine()
```

```
5 end
6
7 // Utility-Rechner
8 macro Calculator()
9     var result = 42 * 2
10     Message("Ergebnis: " + result)
11 end
```

3.2 Makro-Manager

Der integrierte Makro-Manager ermöglicht:

- Aufzeichnen von Makros
- Binden an Hotkeys
- Erstellen, Verwalten und Bearbeiten von .mrmac-Dateien
- Direktes Editieren innerhalb des Managers

Kapitel 4

Editieren wie ein Profi

4.1 Umgang mit großen Dateien

MR nutzt fortschrittliche Datenstrukturen:

Technologie	Zweck	Beispiel
Piecetables	Effiziente Textmanipulation	Bearbeitung > RAM-Größe
Addbuffers	Schnelles Einfügen/Anhängen	Streaming-Blöcke
Tries	Präfix-Suche	Autovervollständigung

Performance: MR lädt 1 GB Text in unter 1 Sekunde und indiziert den gesamten Text in unter 800 Millisekunden – auch auf durchschnittlicher Hardware.

4.2 Blocktypen

MR unterstützt drei Blockmarkierungs-Modi:

1. **Stream Block** – Kontinuierlicher Textfluss
2. **Line Block** – Zeilenweise Markierung
3. **Column Block** – Spaltenweise Markierung (rechteckig)

Sortierung ist für alle markierten Blöcke verfügbar.

4.3 Code-Faltung und Smart-Indenting

- **Automatische mehrstufige grafische Code-Faltung**
- **Smart Indenting/Undenting** – Kontextabhängig
- **Sprachautomatik** – Erkennt automatisch die Programmiersprache

4.4 Die Minimap

Die Sub-Character-Minimap bietet eine verkleinerte Übersicht des gesamten Dokuments und ermöglicht schnelle Navigation in großen Dateien.

Kapitel 5

Fenster-Management

5.1 Der Fenster-Manager

MR's Fenster-Manager beherrscht:

- **Tiling** – Gleichmäßige Aufteilung
- **Minimieren** – Fenster in Taskleiste ablegen
- **Cascading** – Überlappende Anordnung

5.2 Virtuelle Desktops

Wechseln Sie zwischen komplett unabhängigen Arbeitsumgebungen – jede mit eigenen Fenstern, Makros und Einstellungen.

5.3 Inter-Window Copy & Paste

Kopieren und Einfügen funktioniert:

- Zwischen MR-Fenstern
- Zwischen MR und dem Betriebssystem (System-Clipboard)

Kapitel 6

Workspaces und Projekte

6.1 Workspace-Management

- **Speichern/Laden** von Workspaces
- **Autoload** – Workspace beim Start automatisch laden
- **Autosave** – Automatisches Speichern in Intervallen
- **Workspace-weite Suche** – Multifile-Suche & Ersetzung

6.2 Profile

Profile können pro Dateiendung oder für Gruppen von Dateiendungen definiert werden:

- Syntax-Highlighting-Regeln
- Einrückungseinstellungen
- Compiler-Einstellungen
- Farbthemen

6.3 Farbthemen

- Laden und Speichern von Farbschemata
- Verknüpfung mit Dateiendungsprofilen
- Import/Export von Themes

Kapitel 7

Suche und Ersetzung

7.1 Reguläre Ausdrücke

MR verwendet **PCRE2** (Perl Compatible Regular Expressions) – eine der mächtigsten Regex-Implementierungen.

```
1 // Beispiel: Finde alle Funktionsdefinitionen
2 ^(.*\s+)?(\w+)\s*(.*)\s*\{
3
4 // Beispiel: Ersetze mit Capture Groups
5 Suchen:  (\w+)\.oldMethod\(\s*
6 Ersetzen: $1.newMethod(\s*
```

7.2 Rekursive Multifile-Suche

Durchsuchen Sie ganze Verzeichnisbäume mit:

- Dateifiltern (*.cpp, *.h, etc.)
- Regex-Unterstützung
- Vorschau der Fundstellen
- Ersetzung über mehrere Dateien mit Undo-Funktion

Kapitel 8

Compiler-Integration

8.1 Compiler-Profile

Richten Sie Compiler für Ihre Projekte ein:

```
1 # Beispiel: GCC-Profil  
2 Compiler: gcc  
3 Flags: -Wall -Wextra -std=c11  
4 Output-Parsing: Fehlerformat GNU
```

8.2 Fehler-Tracking

- Automatisches Parsen der Compiler-Ausgabe
- Sprung zur Fehlerstelle per Tastendruck
- Fehlerliste mit Schweregrad
- Unterstützung für make, cmake und direkte Compiler-Aufrufe

Kapitel 9

LSP-Integration

9.1 Language Server Protocol

MR unterstützt das Language Server Protocol für:

- Autovervollständigung
- Inline-Fehleranzeige
- Go-to-Definition
- Find-References
- Hover-Informationen
- Refactoring

9.2 LSP-Konfiguration

Konfigurieren Sie LSP-Server in den Profileinstellungen oder pro Workspace.

Kapitel 10

Tastatur und Keymapping

10.1 Key-Mapping-Manager

Der integrierte Key-Mapping-Manager ermöglicht:

- Komplette Neubelegung aller Tasten
- Laden vorgefertigter Mappings
- Emulation anderer Editoren

10.2 Emulationsmodi

MR kann andere **stateless** Editoren emulieren:

- Emacs
- Nano
- WordStar
- ...und weitere

Wichtig: Die Emulation betrifft nur stateless Editoren. Stateful-Editoren wie Vim (Modal-Editing) werden nicht vollständig emuliert.

Kapitel 11

Erweiterte Funktionen

11.1 Drucken und Export

- Drucken über PDF-Export
- Syntax-Highlighting im Export erhalten
- Konfigurierbare Seiteneinstellungen

11.2 Pipes und Shell-Kommandos

Dateien können direkt aus der Ausgabe von Shell-Kommandos bezogen werden:

```
1 # In MR: Datei aus Pipe öffnen  
2 Ctrl+R, dann: ls -la | grep .cpp
```

11.3 Automatische Sprachdetektion

MR erkennt automatisch:

- Programmiersprache anhand Dateiendung
- Shebang-Zeilen (`#!/bin/python`, etc.)
- Modelines
- Coding-Header

Kapitel 12

Tastaturreferenz (Auszug)

Tastenkombination	Funktion
Ctrl+O	Datei öffnen
Ctrl+S	Speichern
Ctrl+F	Suchen
Ctrl+H	Suchen & Ersetzen
Ctrl+Z	Rückgängig
Ctrl+Y	Wiederherstellen
Ctrl+C / Ctrl+V	Kopieren / Einfügen
Alt+1-9	Desktop wechseln
F5	Makro aufzeichnen
F6	Makro-Manager
Ctrl+Space	Autovervollständigung

Tabelle 12.1: Häufigste Tastenkombinationen (Standardbelegung)

Kapitel 13

Fehlerbehebung

13.1 Häufige Probleme

MR startet nicht

```
1 # Prüfen Sie die Terminal-Fähigkeiten
2 echo $TERM
3 # Sollte xterm-256color oder ähnlich sein
4
5 # ncursesw prüfen
6 ldd ./mr | grep ncurses
```

UTF-8-Zeichen werden nicht korrekt dargestellt

```
1 # Locale prüfen
2 locale
3 # Ggf. setzen:
4 export LANG=de_DE.UTF-8
```

Langsame Performance bei sehr großen Dateien

- Minimap temporär deaktivieren
- Syntax-Highlighting für diese Datei reduzieren
- Makro-Coprozessor-Auslastung prüfen

Kapitel 14

Technische Architektur

14.1 Verwendete Technologien

- **TVISION:** Turbo Vision C++ Rewrite von magiblot (GitHub) – Terminal-UI-Framework
- **ncursesw:** Terminal-Steuerung mit Wide-Character-Support
- **Piecetables:** Datenstruktur für effiziente Textmanipulation
- **Tries:** Präfixbäume für schnelle Textsuchen
- **MRMAC Compiler:** In-RAM Bytecode-Kompilierung

14.2 Der Coprozessor im Detail

Der Coprozessor arbeitet mit mehreren parallelen Compute-Lanes:

1. **I/O-Lane:** Dateizugriffe, Netzwerk, Pipes
2. **COMPUTE-Lane:** CPU-intensive Berechnungen
3. **MACRO-Lane:** MRMAC-Makroausführung
4. **MINIMAP-Lane:** Kontinuierliche Minimap-Aktualisierung

Dies ermöglicht:

- Makro-Ausführung während der Benutzer weiter editiert
- Mehrere Makros parallel in verschiedenen Fenstern
- Mehrere Makros im selben Fenster
- Kombination aller Modi

Anhang A

MRMAC-Sprachreferenz (Kurzfassung)

A.1 Grundlegende Syntax

```
1 // Kommentar
2 /* Block-Kommentar */
3
4 // Variablen
5 var zahl = 42
6 var text = "Hallo"
7
8 // Funktionen
9 macro MeineFunktion(param1, param2)
10     // Code
11     return wert
12 end
13
14 // Bedingungen
15 if bedingung then
16     // Code
17 else
18     // Code
19 end
20
21 // Schleifen
22 while bedingung do
23     // Code
24 end
25
26 for i = 1 to 10 do
27     // Code
28 end
```

A.2 Editor-Kommandos (Auswahl)

```
1 InsertText("text")
2 DeleteLine()
```



```
3 MoveCursor(zeile, spalte)
4 SelectBlock(start, ende)
5 SearchPattern("regex")
6 ReplacePattern("such", "ersatz")
7 SaveFile()
8 OpenFile("pfad")
```

Anhang B

Lizenz und Danksagungen

B.1 Zitate, die uns inspirierten

„C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do it blows your whole leg off.“

— Bjarne Stroustrup

„Talk is cheap. Show me the code.“

— Linus Torvalds

„Debugging is twice as hard as writing a program in the first place.“

— Brian W. Kernighan

B.2 Projekt

MR ist Open Source und verfügbar unter:

<https://github.com/ebeneezer/MR>

„Software is hard. It's harder than anything else I've ever had to do.“

— Donald Knuth